

## **Program 1: Write a program to calculate the mod exponent of a big number.**

### **Code:**

```
# PROGRAM 1: Modular Exponentiation of a Big Number
```

```
def mod_exp(base, exponent, modulus):  
    result = 1  
    base = base % modulus  
    while exponent > 0:  
        if exponent % 2 == 1:    # If exponent is odd  
            result = (result * base) % modulus  
            base = (base * base) % modulus  
            exponent //= 2  
    return result  
  
# ---- Main Driver ----  
base = int(input("Enter base: "))  
exponent = int(input("Enter exponent: "))  
modulus = int(input("Enter modulus: "))  
print("\nResult =", mod_exp(base, exponent, modulus))
```

### **Output:**

**Enter base: 5**

**Enter exponent: 13**

**Enter modulus: 7**

**Result = 5**

**Program-2: Write a program to demonstrate RSA algorithm by taking input plaintext from a file and produce outfile into ciphertext file.**

**Code:**

```
# PROGRAM 2: RSA Algorithm using File Input and File Output
```

```
def gcd(a, b):
```

```
    while b != 0:
```

```
        a, b = b, a % b
```

```
    return a
```

```
def mod_inverse(e, phi):
```

```
    # Extended Euclid Algorithm
```

```
    old_r, r = e, phi
```

```
    old_s, s = 1, 0
```

```
    while r != 0:
```

```
        q = old_r // r
```

```
        old_r, r = r, old_r - q * r
```

```
        old_s, s = s, old_s - q * s
```

```
    if old_s < 0:
```

```
        old_s += phi
```

```
    return old_s
```

```
def rsa_encrypt_char(ch, e, n):
```

```

        return pow(ord(ch), e, n)

def rsa_decrypt_char(c, d, n):
    return chr(pow(c, d, n))

# -----
# Main Program
# -----

p = int(input("Enter prime p: "))
q = int(input("Enter prime q: "))
n = p * q
phi = (p - 1) * (q - 1)
print("\nChoose encryption exponent e such that gcd(e, phi)=1")
e = int(input("Enter e: "))
if gcd(e, phi) != 1:
    print("Invalid e! Must be coprime with  $\phi(n)$ .")
    exit()
d = mod_inverse(e, phi)
print("\nPublic Key =", (e, n))
print("Private Key =", d)
# File input/output
infile = "input.txt"
outfile = "rsa_output.txt"

with open(infile, "r") as f:

```

```
plaintext = f.read()
cipher_nums = []
for ch in plaintext:
    cipher_nums.append(str(rsa_encrypt_char(ch, e, n)))
with open(outfile, "w") as f:
    f.write(" ".join(cipher_nums))
print("\nRSA Encryption Completed!")
print("Ciphertext saved in:", outfile)
```

### **Output:**

**Enter prime p: 7**

**Enter prime q: 11**

**Choose encryption exponent e such that  $\gcd(e, \phi)=1$**

**Enter e: 7**

**Public Key = (7, 77)**

**Private Key = 43**

**RSA Encryption Completed!**

**Ciphertext saved in: rsa\_output.txt**

**Plain Text message: my name is meet shah**

**RSA Output file encryption : 32 44 32 33 48 32 73 32 63 3 32 32 73 73 74 32 3  
69 48 69**

### **Program-3: Write a program to implement Fermat Little Theorem.**

#### **Code:**

```
# PROGRAM 3: Fermat's Little Theorem Demonstration
```

```
def mod_exp(base, exp, mod):
```

```
    result = 1
```

```
    base = base % mod
```

```
    while exp > 0:
```

```
        if exp % 2 == 1:
```

```
            result = (result * base) % mod
```

```
            base = (base * base) % mod
```

```
            exp //= 2
```

```
    return result
```

```
print("Fermat's Little Theorem Checker")
```

```
a = int(input("Enter value of a: "))
```

```
p = int(input("Enter prime number p: "))
```

```
if a % p == 0:
```

```
    print("gcd(a,p)  $\neq$  1  $\rightarrow$  Fermat's theorem does not apply.")
```

```
else:
```

```
    value = mod_exp(a, p - 1, p)
```

```
    print(f"\nAccording to Fermat's Little Theorem:")
```

```
    print(f"{a}^{p-1} mod {p} = {value}")
```

```
if value == 1:  
    print("The theorem is VERIFIED ✓")  
else:  
    print("The theorem FAILED ✗")
```

**Output:**

**Fermat's Little Theorem Checker**

**Enter value of a: 6**

**Enter prime number p: 7**

**According to Fermat's Little Theorem:**

$$6^{(p-1)} \bmod 7 = 1$$

**The theorem is VERIFIED ✓**